Chapitre 5

NumPy et SciPy

Calcul numérique et fondations mathématiques pour le machine learning

Calculer avec NumPy et SciPy

Ce chapitre utilise des tableaux numériques pour résoudre des problèmes de mesure, de vente et de probabilité.

1. Représenter les données et effectuer des calculs vectorisés.
2. Résoudre un système d'équations et ajuster une droite.
3. Calculer une dérivée, un minimum et une intégrale.
4. Décrire une loi de probabilité et simuler des observations.

Les définitions et les formules précèdent les exercices. Le notebook reprend les exemples avec les mêmes numéros de diapositives.

Préparer le notebook

Ouvrir un notebook Jupyter ou l'environnement Python du cours. Si les bibliothèques manquent, exécuter dans le **terminal du même environnement** :

```
python -m pip install numpy scipy matplotlib
```

Dans le notebook, exécuter :

```
import numpy as np
import scipy
from scipy import linalg, optimize, integrate, stats
print("NumPy", np.__version__, "SciPy", scipy.__version__)
```

`as np` donne un nom court au module. `from scipy import ...` charge les familles d'outils que l'on utilisera. Les graphiques du notebook sont fournis avec leur code Matplotlib.

NumPy et SciPy : qui fait quoi ?

Outil	Utilisation
NumPy	Représenter des nombres en tableaux et calculer sur des ensembles de valeurs
`scipy.linalg`	Résoudre des équations et ajuster un modèle linéaire
`scipy.optimize`	Chercher les paramètres qui minimisent une erreur
`scipy.integrate`	Calculer une quantité accumulée, par exemple une distance
`scipy.stats`	Décrire des lois de probabilité et calculer des probabilités

Ces outils permettent de préparer des données, d'ajuster un modèle et d'interpréter ses résultats. Nous retrouverons ces calculs en machine learning.

Scalaire, vecteur, matrice : des formes différentes

Un **scalaire** est un nombre. Un **vecteur** est une suite de composantes. Une **matrice** range des nombres en lignes et en colonnes.

```
x = np.array([10., 20., 30.])
A = np.array([[1., 2.], [3., 4.], [5., 6.]])
print(x.shape, x.ndim)  # (3,) et 1
print(A.shape, A.ndim)  # (3, 2) et 2
print(A.dtype)          # float64
```

`shape` donne la longueur de chaque axe ; `ndim` donne le nombre d'axes. `dtype` donne le type commun aux cases. Pour conserver les décimales, utiliser `dtype=float` ou des valeurs comme `1.0`.

En deep learning, on parle souvent de **tenseur** pour un tableau à plusieurs axes : par exemple `(nombre_images, hauteur, largeur, canaux)`.

Source : NumPy : tableaux et attributs

Créer les tableaux dont on a besoin

```
print(np.arange(4))         # [0 1 2 3]
print(np.linspace(0, 1, 5)) # [0. 0.25 0.5 0.75 1.]
print(np.zeros((2, 3)))     # 2 lignes, 3 colonnes de zéros
print(np.ones(3))           # [1. 1. 1.]
b = np.arange(6).reshape(2, 3)
print(b)                   # [[0 1 2], [3 4 5]]
```

`arange` fixe un pas ; la borne de fin est exclue. `linspace` fixe le nombre de points et inclut les deux bornes par défaut.

`reshape` change la disposition sans changer le nombre de valeurs. Six valeurs peuvent être rangées en `(3, 2)`, car $3 \times 2 = 6$. La forme `(4, 2)` en exige huit : elle est incompatible avec ce tableau.

Lire, filtrer et copier

```
A = np.array([[10., 20.], [30., 40.], [50., 60.]])
print(A[1, 0])       # 30.0 : ligne 1, colonne 0
print(A[:, 1])       # [20. 40. 60.] : toute la colonne 1
x = np.array([10., 20., 30.])
print(x[x >= 20])    # [20. 30.] : sélection par condition
essai = x[:2].copy()
essai[0] = 999       # x reste inchangé
```

Les indices commencent à zéro. `:` prend toutes les positions d'un axe ; dans `[:2]`, la borne 2 est exclue. Un **masque** contient un booléen par case testée.

Une tranche simple partage les données avec l'original : c'est une **vue**. `copy()` crée des données indépendantes. Sans cette copie, la dernière affectation modifierait `x`.

Source : NumPy : vues et copies

Les opérations élément par élément

```
x = np.array([1., 2., 3.])
print(x ** 2)       # [1. 4. 9.]
print(2 * x + 1)    # [3. 5. 7.]
print(np.sqrt(x))   # racine carrée de chaque case
print(np.exp(x))    # exponentielle de chaque case
print(np.log(x))    # logarithme naturel, x > 0 ici
```

`exp(x)` calcule e^x, où $e \simeq 2{,}718$. Le logarithme naturel est son opération inverse : $\log(e^x) = x$.

Une fonction mathématique appliquée à chaque case, comme `np.sqrt`, est une **fonction universelle** de NumPy, ou *ufunc*. `*` multiplie case par case ; `**` élève à une puissance.

La vectorisation

Vectoriser, c'est exprimer le calcul sur un tableau entier. NumPy exécute les boucles numériques dans du code compilé, sur des données de même type.

```
valeurs = [1., 2., 3.]
resultat_python = []
for valeur in valeurs:
    resultat_python.append(2 * valeur + 1)

x = np.array(valeurs)
resultat_numpy = 2 * x + 1
print(resultat_python, resultat_numpy)
```

Les deux versions produisent 3, 5 et 7. Elles effectuent un travail en $O(n)$ pour n valeurs. Le gain attendu concerne le temps réel ; une expression courte n'est pas automatiquement en $O(1)$.

Exercice : convertir des mesures

Un capteur donne des valeurs brutes. La formule de conversion est fournie :

$$T = 0{,}5r - 10,$$

avec r la valeur brute et T la température en degrés Celsius.

```
brut = np.array([20., 40., 60., 80.])
```

Travail demandé

1. Prévoir les quatre températures à la main.
2. Calculer tout le tableau sans écrire de boucle.
3. Sélectionner les températures strictement supérieures à 15 degrés.

Pour vérifier la méthode, écrire en une phrase le calcul fait pour chaque case.

Résultat : quatre conversions en une expression

```
temperature = 0.5 * brut - 10
chaudes = temperature[temperature > 15]
print(temperature)  # [ 0. 10. 20. 30.]
print(chaudes)      # [20. 30.]
assert np.allclose(temperature, [0, 10, 20, 30])
```

`allclose` vérifie que les nombres correspondants sont proches, avec une tolérance, ce qui convient aux calculs flottants soumis aux arrondis.

Avec le coefficient 0,6, la conversion donne `[2., 14., 26., 38.]`. La valeur attendue dans le test doit correspondre au coefficient utilisé.

Vitesse et mémoire : ce qu'il faut vérifier

La conversion d'une liste et les tableaux temporaires ont un coût en temps ou en mémoire.

```
import timeit
x_test = np.linspace(0, 100, 100_000)
liste_test = x_test.tolist()
tp = timeit.repeat("[0.5*v-10 for v in liste_test]",
                   globals=globals(), number=10, repeat=3)
tn = timeit.repeat("0.5*x_test-10", globals=globals(),
                   number=10, repeat=3)
print(min(tp) / 10, min(tn) / 10)  # secondes par calcul
```

Code fourni : trois mesures de dix calculs. `globals()` donne accès aux variables. Conversion hors mesure.

Attention : `np.vectorize` repose essentiellement sur une boucle Python ; ce nom ne garantit aucun gain de vitesse.

Source : NumPy : vectorize

Les axes donnent un sens aux résumés

```
X = np.array([[10., 100.],
              [20., 200.],
              [30., 300.]])
print(X.mean(axis=0))  # [20. 200.] : moyenne par colonne
print(X.sum(axis=1))   # [110. 220. 330.] : somme par ligne
print(X.shape)        # (3, 2)
```

Une ligne représente une observation ; une colonne représente une variable. `axis=0` réduit les lignes et garde une valeur par colonne. `axis=1` réduit les colonnes et garde une valeur par ligne.

Pour 100 observations et 4 variables, `mean(axis=0)` renvoie 4 moyennes : le résultat a la forme `(4,)`. Additionner des colonnes n'a de sens que si les quantités et leurs unités le permettent.

Le broadcasting : associer des formes compatibles

On veut soustraire à chaque colonne sa moyenne :

$$\begin{pmatrix} 10 & 100 \\ 20 & 200 \\ 30 & 300 \end{pmatrix} - \begin{pmatrix} 20 & 200 \end{pmatrix} = \begin{pmatrix} -10 & -100 \\ 0 & 0 \\ 10 & 100 \end{pmatrix}.$$

NumPy applique la même paire de moyennes à chaque ligne. C'est le **broadcasting**.

```
moyennes = X.mean(axis=0)   # forme (2,)
centre = X - moyennes       # forme (3, 2)
print(centre)
```

Depuis la droite, deux dimensions doivent être égales ou l'une doit valoir 1 ; un axe absent compte comme 1. Un facteur par ligne nécessiterait ici la forme `(3, 1)`, obtenue avec `reshape(3, 1)`.

Source : NumPy : broadcasting

Moyenne, variance et écart-type

Pour une série $x_1, \ldots, x_n$, la notation $\sum$ signifie « additionner » :

$$\mu = \frac{1}{n}\sum_i x_i, \qquad v = \frac{1}{n}\sum_i (x_i - \mu)^2, \qquad \sigma = \sqrt{v}.$$

La moyenne donne un niveau central ; l'écart-type mesure la dispersion et conserve l'unité des observations.

```
x = np.array([10., 20., 30.])
print(x.mean())       # 20.0
print(x.var(ddof=0))  # environ 66.6667
print(x.std(ddof=0))  # environ 8.1650
```

`ddof=0` divise par n : on décrit cette série. `ddof=1` divise par $n - 1$, notamment pour estimer une variance à partir d'un échantillon indépendant.

Source : NumPy : std

Exercice : standardiser deux variables

Une variable varie autour de 20, l'autre autour de 200. Pour les comparer dans des unités standardisées :

$$Z_{ij} = \frac{X_{ij} - \mu_j}{\sigma_j}.$$

j indique la colonne. On soustrait sa moyenne et on divise par son écart-type. **Travail demandé :** calculer `Z` pour la matrice `X` vue précédemment.

Étapes : moyenne par colonne, écart-type par colonne, soustraction, division. Prévoir la forme de chaque résultat avant d'exécuter.

Si une colonne est constante, son écart-type vaut zéro. Convention pour l'exemple : le remplacer par 1 pour la division. `np.where(condition, a, b)` choisit `a` là où la condition est vraie, `b` ailleurs.

Résultat : centrer et réduire chaque colonne

```
mu = X.mean(axis=0)
sigma = X.std(axis=0, ddof=0)
echelle = np.where(sigma == 0, 1., sigma)
Z = (X - mu) / echelle
print(Z)
# [[-1.2247, -1.2247], [0., 0.], [1.2247, 1.2247]] environ
print(Z.mean(axis=0))  # proche de [0., 0.]
print(Z.std(axis=0))   # [1., 1.] ici
```

Les colonnes non constantes ont une moyenne proche de 0 et un écart-type proche de 1. Avec notre convention, une colonne constante devient nulle.

Pour un futur modèle, calculer `mu` et `sigma` sur les données d'entraînement, puis réutiliser ces valeurs sur les nouvelles données. Les recalculer sur le test introduirait une fuite d'information.

Un produit scalaire est une somme pondérée

Une boutique vend des cahiers à 3 euros et des stylos à 5 euros. Pour les quantités $q = (2, 4)$ et les prix $p = (3, 5)$:

$$q \cdot p = 2 \times 3 + 4 \times 5 = 26 \text{ euros.}$$

Le **produit scalaire** multiplie les composantes correspondantes, puis les additionne.

```
q = np.array([2., 4.])
p = np.array([3., 5.])
print(q * p)          # [ 6. 20.] : produits séparés
print(q @ p)          # 26.0 : somme des produits
print(np.sqrt(q @ q)) # longueur du vecteur, environ 4.4721
```

Pour des vecteurs de même longueur, `@` calcule le produit scalaire. La **norme euclidienne** est $\|q\| = \sqrt{q_1^2 + q_2^2}$: une mesure de longueur.

La multiplication matrice-vecteur

Une ligne par journée, une colonne par produit :

$$\underbrace{\begin{pmatrix} 2 & 4 \\ 1 & 3 \\ 5 & 2 \end{pmatrix}}_{Q\ :\ (3,2)} \times \underbrace{\begin{pmatrix} 3 \\ 5 \end{pmatrix}}_{p\ :\ (2,)} = \underbrace{\begin{pmatrix} 26 \\ 18 \\ 25 \end{pmatrix}}_{\text{recettes}\ :\ (3,)} .$$

Chaque ligne produit un résultat par produit scalaire.

```
Q = np.array([[2., 4.], [1., 3.], [5., 2.]])
recettes = Q @ p
print(recettes)   # [26. 18. 25.]
```

Pour deux matrices, les dimensions intérieures doivent coïncider : (n, d) @ (d, k) donne (n, k). En ML, $Xw + b$ utilisera la même opération pour calculer une prédiction par observation.

Source : NumPy : produit matriciel

Transposer : échanger les lignes et les colonnes

```
print(Q.shape)       # (3, 2)
print(Q.T.shape)     # (2, 3)
print(Q.T)
# [[2. 1. 5.]
#  [4. 3. 2.]]
print(p.shape, p.T.shape)  # (2,) et (2,)
colonne = p.reshape(2, 1)
print(colonne.shape)      # (2, 1)
```

`.T` est la transposée. Un vecteur NumPy 1D n'a pas de distinction ligne/colonne : sa transposée garde la forme `(2,)`.

Si X a la forme `(100, 4)`, alors `X.T` a la forme `(4, 100)`. Pour un vecteur r de longueur 100, `X.T @ r` produit donc 4 valeurs. Ce calcul interviendra dans le gradient.

Un système linéaire : retrouver deux prix

Deux factures donnent : deux cahiers et un stylo pour 11 euros ; un cahier et trois stylos pour 18 euros. On cherche le prix c du cahier et s du stylo.

$$\begin{cases} 2c + s = 11 \\ c + 3s = 18 \end{cases} \qquad \Longleftrightarrow \qquad A\theta = b.$$

Chaque ligne de A contient les coefficients d'une équation. θ regroupe les inconnues ; b contient les totaux connus.

Une solution unique exige ici deux informations indépendantes. Répéter la même facture ne permettrait pas de séparer les deux prix. Une matrice carrée possède autant de lignes que de colonnes.

Exercice : retrouver les deux prix

`linalg.solve(A, b)` résout le système carré $A\theta = b$ quand la solution est unique.

```
A_factures = np.array([[2., 1.], [1., 3.]])
b_factures = np.array([11., 18.])
```

Travail demandé

1. Calculer les deux prix avec `solve`.
2. Recalculer les factures par multiplication matricielle.
3. Calculer l'écart aux factures fournies.

Cet écart s'appelle le **résidu** : $r = A\theta - b$. On attend des valeurs proches de zéro, compte tenu des arrondis numériques.

Source : SciPy : linalg.solve

Résultat : vérifier les deux factures

```
theta_factures = linalg.solve(A_factures, b_factures)
print(theta_factures)                    # [3. 5.]
print(A_factures @ theta_factures)       # [11. 18.]
residu = A_factures @ theta_factures - b_factures
print(residu)                            # proche de [0. 0.]
assert np.allclose(residu, 0, rtol=0, atol=1e-10)
```

Un cahier coûte 3 euros, un stylo 5 euros. Vérification à la main : $2 \times 3 + 5 = 11$ et $3 + 3 \times 5 = 18$.

`atol` est la tolérance absolue ; `rtol=0` désactive la tolérance relative. Un système sans solution unique peut déclencher `LinAlgError`. On utilise un solveur directement ; calculer explicitement une matrice inverse est inutile ici.

Quand les données ne donnent pas des égalités exactes

Quatre campagnes publicitaires ont produit ces observations fictives :

Budget x (centaines d'euros)	0	1	2	3
Ventes y (articles)	10	14	15	21

On cherche une droite $\widehat{y} = b + ax$. a est la variation calculée pour une unité de budget supplémentaire ; b est la valeur à budget nul.

$$J(a, b) = \frac{1}{n} \sum_i \big((b + ax_i) - y_i\big)^2.$$

$\widehat{y}$ est une prédiction du modèle. J est l'**erreur quadratique moyenne**, ou MSE : la moyenne des erreurs au carré. On cherche les coefficients qui la minimisent.

Ajuster la droite avec les moindres carrés

On écrit $M\theta \simeq y$, avec $\theta = (b, a)$. La colonne de 1 multiplie la constante b ; la colonne des budgets multiplie a.

```
x_pub = np.array([0., 1., 2., 3.])
y = np.array([10., 14., 15., 21.])
M = np.column_stack((np.ones(4), x_pub))
print(M)  # [[1. 0.], [1. 1.], [1. 2.], [1. 3.]]
```

`column_stack` assemble les vecteurs en colonnes. `linalg.lstsq(M, y)` minimise la somme des erreurs au carré ; sa première sortie contient les coefficients. Minimiser cette somme ou sa moyenne donne ici les mêmes coefficients.

Application : appeler la fonction, récupérer la sortie d'indice 0, puis calculer `M @ theta`. Préciser la forme des coefficients et celle des prédictions.

Source : SciPy : linalg.lstsq

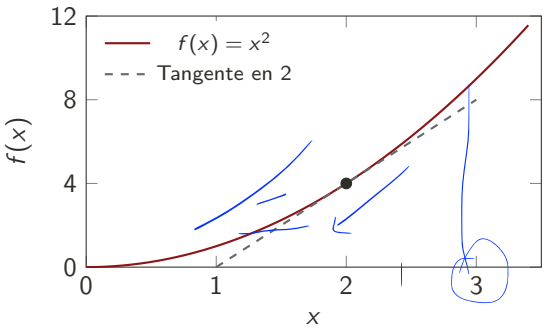
Résultat : une première régression linéaire

```
solution = linalg.lstsq(M, y)
theta = solution[0]
y_calcule = M @ theta
mse = ((y_calcule - y) ** 2).mean()
print(theta)       # [9.9 3.4] : constante, puis pente
print(y_calcule)   # [9.9 13.3 16.7 20.1]
print(mse)         # environ 1.05
```

La droite $\widehat{y} = 9{,}9 + 3{,}4x$ résume les quatre observations. `theta` a la forme `(2,)` ; les prédictions ont la forme `(4,)`.

Une erreur faible sur ces données ne prouve ni une relation causale ni une bonne prédiction sur d'autres campagnes. En ML, on vérifiera aussi le modèle sur des données non utilisées pour l'ajuster.

Une dérivée décrit une variation locale



La dérivée $f'(x)$ est la pente de la tangente : elle décrit la variation de f pour une petite variation de x.

Pour $f(x) = x^2$, la formule est $f'(x) = 2x$. À $x = 2$, la pente vaut 4.

Donc $f(2 + h) \approx f(2) + 4h$ pour un petit h.

Approcher une dérivée avec deux évaluations

Sans formule de dérivée, on peut utiliser un petit pas h :

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}.$$

```
def f(x):
    return x ** 2

h = 1e-5
pente_locale = (f(2 + h) - f(2 - h)) / (2 * h)
print(pente_locale)  # proche de 4
```

En $x = 3$, la même approximation donne une valeur proche de $f'(3) = 6$. Le pas doit être assez petit pour décrire la variation locale, mais pas si petit que les arrondis dominent la soustraction.

Il s'agit d'une approximation numérique. Les dérivées analytiques ou le calcul automatique des frameworks seront d'autres moyens d'obtenir ces variations.

La dérivée indique dans quel sens descendre

On veut minimiser $J(w) = (w-3)^2$. Le minimum vaut zéro en $w = 3$. Sa dérivée est $J'(w) = 2(w-3)$.

$$w_{\text{nouveau}} = w - \eta J'(w).$$

$\eta > 0$ est le **pas d'apprentissage** : il règle la taille du déplacement.

- Une dérivée positive incite à diminuer w.
- Une dérivée négative incite à augmenter w.

Avec $w = 0$ et $\eta = 0{,}2$, la dérivée vaut -6 ; le nouveau w vaut $0 - 0{,}2(-6) = 1{,}2$. Cette mise à jour est la **descente de gradient** en une dimension.

Exercice : trois pas de descente

Compléter le calcul de la dérivée et la mise à jour. Tracer les valeurs de `w` avant d'exécuter.

```
w = 0.0
eta = 0.2
for tour in range(3):
    derivee = ...
    w = ...
    print(w, (w - 3) ** 2)
```

Pseudocode : calculer la dérivée au point courant, soustraire le pas multiplié par cette dérivée, puis recommencer au nouveau point.

Formules disponibles : $J'(w) = 2(w - 3)$ et $w_{\text{nouveau}} = w - \eta J'(w)$. Le résultat doit-il atteindre exactement 3 après trois tours ?

Résultat : la perte diminue à chaque pas

```
w = 0.0
eta = 0.2
for tour in range(3):
    derivee = 2 * (w - 3)
    w = w - eta * derivee
    print(w, (w - 3) ** 2)
```

Tour	w après mise à jour	$J(w)$
1	1,2	3,24
2	1,92	1,1664
3	2,352	0,419904

On se rapproche de 3 sans l'atteindre exactement. Un pas trop grand peut provoquer des oscillations ou une divergence. Avec $\eta = 1{,}1$, l'écart à 3 grandit à chaque tour dans cet exemple.

Plusieurs paramètres : le gradient

Une **dérivée partielle** mesure la variation d'une fonction quand on change un paramètre en gardant les autres fixes. Le **gradient** regroupe ces dérivées dans un vecteur.

Pour la droite, $\theta = (b, a)$, $r = M\theta - y$ et $J = \frac{1}{n}\sum_i r_i^2$:

$$\frac{\partial J}{\partial b} = \frac{2}{n}\sum_i r_i, \qquad \frac{\partial J}{\partial a} = \frac{2}{n}\sum_i x_i r_i, \qquad \nabla J = \frac{2}{n} M^T r.$$

`M.T @ r` produit ici deux composantes, une par coefficient. La mise à jour devient $\theta_{\text{nouveau}} = \theta - \eta \nabla J$.

Chaque composante du gradient décrit la variation de la perte par rapport à un paramètre. Le pas η règle l'amplitude de la mise à jour.

SciPy peut chercher les paramètres

`optimize.minimize` attend une fonction renvoyant **un nombre à minimiser** et un point de départ `x0`. Le paramètre `jac` reçoit la fonction qui calcule le gradient.

```
def perte(theta):
    r = M @ theta - y
    return (r ** 2).mean()

def gradient(theta):
    r = M @ theta - y
    return (2 / len(y)) * (M.T @ r)
```

On donnera ensuite `perte` et `gradient` comme objets fonctions, sans les appeler nous-mêmes. `method="BFGS"` choisit un algorithme d'optimisation : il utilise le gradient, mais ne se réduit pas à notre boucle à pas fixe.

Source : SciPy : optimize.minimize

Résultat : comparer deux méthodes de résolution

```
resultat = optimize.minimize(perte, x0=np.array([0., 0.]),
                             jac=gradient, method="BFGS")
print(resultat.success, resultat.message)
print(resultat.x)    # proche de [9.9 3.4]
print(resultat.fun)  # proche de 1.05
assert resultat.success
assert np.allclose(resultat.x, theta, atol=1e-6)
```

`success` indique si l'algorithme a satisfait ses critères d'arrêt ; `message` les précise. `x` contient les paramètres trouvés ; `fun` contient la valeur de la perte.

Ici, les deux méthodes retrouvent le même minimum d'un problème quadratique à solution unique. Pour une fonction quelconque, la convergence ne garantit pas un minimum global.

La règle de chaîne : dériver une composition

Si $z = f(w)$ puis $L = g(z)$, une variation de w agit sur L à travers z :

$$\frac{dL}{dw} = \frac{dL}{dz}\,\frac{dz}{dw}.$$

Exemple : $z = 2w + 1$, puis $L = (z - 5)^2$.

$$\frac{dz}{dw} = 2, \qquad \frac{dL}{dz} = 2(z - 5), \qquad \frac{dL}{dw} = 4(z - 5).$$

À $w = 1$, $z = 3$ et la dérivée finale vaut -8.

Un réseau de neurones enchaîne des fonctions. La **rétropropagation** applique cette règle, en remontant les dépendances du calcul, pour obtenir les gradients des paramètres. Les frameworks automatiseront ce travail.

Une intégrale additionne des contributions

Un mobile avance avec la vitesse $v(t) = 2t$ m/s entre 0 et 4 secondes, sans changer de sens. Sa distance est l'aire sous la courbe de vitesse :

$$d = \int_0^4 v(t)\,dt.$$

Le symbole $\int$ représente une accumulation continue. On peut l'approcher en additionnant des petites aires $v(t_i)\Delta t$.

Ici, l'aire est un triangle de base 4 secondes et de hauteur 8 m/s :

$$d = \frac{4 \times 8}{2} = 16 \text{ m}.$$

Ce résultat calculable à la main servira à contrôler l'intégration numérique.

Exercice : calculer une distance avec SciPy

`integrate.quad(f, a, b)` approche l'intégrale de la fonction `f` entre les bornes `a` et `b`. La fonction doit accepter un nombre et renvoyer un nombre.

```
def vitesse(t):
    return 2 * t
```

Travail demandé

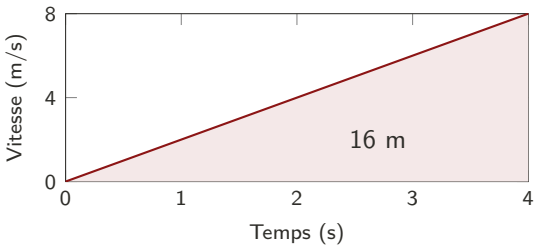
1. Appeler quad entre 0 et 4.
2. Récupérer ses deux sorties : valeur calculée et estimation de l'erreur numérique absolue.
3. Refaire le calcul entre 0 et 3 ; prévoir l'aire du triangle avant l'exécution.

L'estimation de l'erreur concerne le calcul numérique, pas une incertitude sur les mesures ou sur le modèle physique.

Source : SciPy : integrate.quad

Résultat : 16 mètres, puis 9 mètres

```
distance, erreur_numerique = integrate.quad(vitesse, 0, 4)
print(distance, erreur_numerique)  # 16.0 ; erreur très petite
print(integrate.quad(vitesse, 0, 3)[0])  # 9.0
```



Avec seulement des mesures à des instants donnés, on pourrait utiliser des rectangles ou des trapèzes.

Ici, quad choisit lui-même les évaluations de la fonction pour approcher l'intégrale.

Une variable aléatoire et sa loi

Une opération commerciale peut perdre 100 euros, gagner 50 euros ou gagner 200 euros. On note G son bénéfice : c'est une **variable aléatoire**.

Valeur de G (euros)	-100	50	200
Probabilité	0,2	0,5	0,3

La **loi de probabilité** associe les valeurs possibles à leurs probabilités. Chaque probabilité est comprise entre 0 et 1 ; leur somme vaut 1.

Un **événement** est un ensemble de résultats possibles. Ici, « perdre de l'argent » signifie $G < 0$; sa probabilité vaut 0,2, soit 20 %. Ces nombres sont les hypothèses d'un modèle fictif.

Espérance et variance : pondérer par les probabilités

Pour les valeurs g_k de probabilités p_k :

$$\mu = \mathbb{E}[G] = \sum_k p_k g_k, \qquad \mathrm{Var}(G) = \sum_k p_k (g_k - \mu)^2.$$

L'**espérance** est la moyenne théorique du modèle ; la variance mesure sa dispersion.

```
gains = np.array([-100., 50., 200.])
probabilites = np.array([0.2, 0.5, 0.3])
esperance = probabilites @ gains
variance = probabilites @ ((gains - esperance) ** 2)
print(esperance, np.sqrt(variance))  # 65.0 et 105.0 euros
```

Un bénéfice moyen positif ne garantit pas un gain à chaque opération. On retrouve les opérations déjà vues : produit scalaire, calcul par case, racine carrée.

Bernoulli et binomiale

Une variable de **Bernoulli** vaut 1 si un événement a lieu, 0 sinon. Son paramètre p est la probabilité de l'événement.

Si l'on répète n essais **indépendants**, avec la même probabilité p, le nombre K de succès suit une **loi binomiale** :

$$P(K = k) = \binom{n}{k} p^k (1-p)^{n-k}.$$

$\binom{n}{k}$ compte les façons de placer les k succès parmi n essais. « Indépendants » signifie ici que le résultat d'un essai ne change pas les probabilités des autres.

Exemple : dix visiteurs achètent chacun avec probabilité 0,2, indépendamment selon ce modèle. Le nombre d'achats est compris entre 0 et 10. L'espérance du nombre d'achats vaut $np = 2$.

Exercice : calculer des probabilités d'achat

`stats.binom(n=10, p=0.2)` crée un objet représentant la loi binomiale.

Méthode	Signification
`loi.pmf(k)`	$P(K = k)$: exactement k achats
`loi.cdf(k)`	$P(K \leq k)$: au plus k achats
`loi.sf(k)`	$P(K > k)$: strictement plus de k achats

Travail demandé : calculer la probabilité d'aucun achat, d'au moins un achat, puis d'au plus deux achats.

À la main, aucun achat a pour probabilité $(1 - 0{,}2)^{10}$. Pour « au moins un », utiliser le complément de « aucun » : $1 - P(K = 0)$, ou directement `sf(0)`.

Source : SciPy : loi binomiale

Résultat : un événement, une méthode

```
loi_achats = stats.binom(n=10, p=0.2)
print(loi_achats.pmf(0))  # environ 0.1074
print(loi_achats.sf(0))   # environ 0.8926
print(loi_achats.cdf(2))  # environ 0.6778
```

Aucun achat : 10,74 %. Au moins un achat : 89,26 %. Au plus deux achats : 67,78 %.

Avec 20 visiteurs, la probabilité d'au moins un achat vaut $1 - 0{,}8^{20}$, soit environ 98,85 %.

`sf` calcule directement la probabilité de dépassement et évite certaines pertes de précision de `1 - cdf`. Si les décisions des visiteurs sont dépendantes, la loi binomiale peut être inadaptée.

Une loi continue : la loi normale

On modélise un temps d'exécution T en millisecondes par une loi normale de moyenne $\mu = 100$ et d'écart-type $\sigma = 15$.

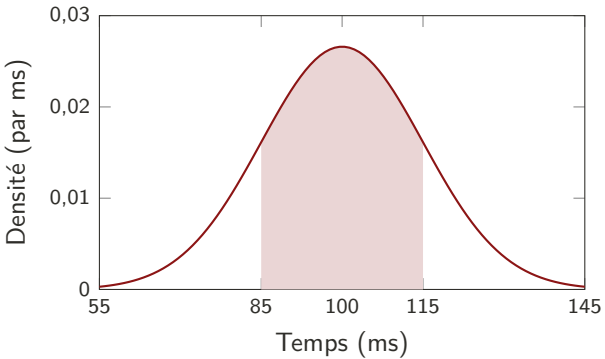
La **densité** décrit comment la probabilité se répartit :

$$f(t) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{t-\mu}{\sigma}\right)^2\right).$$

Cette formule est fournie pour donner le sens des paramètres ; SciPy la calcule pour nous.

Une probabilité correspond à une **aire sous la densité**. La hauteur $f(t)$ n'est pas $P(T = t)$: pour cette loi continue, une valeur exacte a une probabilité nulle. La loi normale est une hypothèse à vérifier sur des mesures réelles.

Une probabilité est une aire



La fonction de répartition est $F(t) = P(T \leq t)$.

Pour un intervalle :

$$P(a \leq T \leq b) = F(b) - F(a).$$

L'aire colorée représente ici la probabilité d'un temps compris entre 85 et 115 ms.

La totalité de l'aire, sur toute la droite réelle, vaut 1.

Exercice : une probabilité et un seuil

Avec `loi = stats.norm(loc=100, scale=15)`, `loc` est la moyenne et `scale` l'**écart-type**, pas la variance.

Méthode	Ce qu'elle renvoie
`pdf(t)`	La hauteur de la densité au point t
`cdf(t)`	$P(T \leq t)$
`sf(t)`	$P(T > t)$
`ppf(q)`	Le seuil t tel que $P(T \leq t) = q$

Travail demandé

1. Calculer $P(85 \leq T \leq 115)$.
2. Trouver le seuil sous lequel tombent 95 % des temps du modèle.

Le deuxième résultat est un **quantile** : une valeur en millisecondes, pas une probabilité.

Source : SciPy : loi normale

Résultat : une probabilité et un seuil

```
loi_temps = stats.norm(loc=100, scale=15)
proba_intervalle = loi_temps.cdf(115) - loi_temps.cdf(85)
seuil_95 = loi_temps.ppf(0.95)
print(proba_intervalle)  # environ 0.6827
print(seuil_95)          # environ 124.67 ms
print(loi_temps.sf(seuil_95))  # environ 0.05
```

Le premier nombre est une proportion. Le second est une durée : environ 95 % des temps du modèle sont inférieurs à 124,67 ms.

La fonction de répartition évaluée au quantile à 95 % retrouve 0,95, à l'arrondi près. Calculer `pdf(100)` ne donnerait pas la probabilité d'un temps exactement égal à 100 ms.

Simuler pour retrouver une probabilité

Un **générateur pseudo-aléatoire** produit des tirages. La graine permet de refaire l'expérience dans le même environnement ; elle ne supprime pas les fluctuations d'échantillonnage.

```
rng = np.random.default_rng(2026)
temps = loi_temps.rvs(size=10_000, random_state=rng)
dans_intervalle = (temps >= 85) & (temps <= 115)
print(temps.mean(), temps.std())
print(dans_intervalle.mean())  # proche de 0.6827
```

`rvs` tire des valeurs de la loi. Le symbole `&` combine deux conditions case par case ; les parenthèses sont nécessaires. La moyenne du masque est une fréquence : `True` compte pour 1, `False` pour 0.

Augmenter le nombre de tirages réduit généralement les fluctuations, sans garantir que chaque nouvelle estimation soit plus proche de la valeur théorique.

Les opérations que l'on retrouvera en ML

Étape	Calcul et signification
Données	X : lignes = observations, colonnes = variables
Préparation	Centrer et réduire avec les statistiques d'entraînement
Prédiction	$Xw + b$: un score par observation
Perte	Un nombre mesurant l'écart entre prédictions et valeurs attendues
Apprentissage	Ajuster les paramètres pour diminuer la perte
Évaluation	Mesurer la qualité sur des observations tenues à l'écart de l'ajustement

Une perte faible sur l'entraînement peut venir d'une mémorisation des données. On parle alors de **surapprentissage** : le modèle généralise mal à de nouvelles observations.

Un premier calcul de neurone

Un neurone calcule un score $z = Xw + b$, puis lui applique une fonction d'**activation**. La sigmoïde est un exemple :

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

```
X_neurone = np.array([[1., 0.], [0., 1.], [1., 1.]])
poids = np.array([1., -1.])
z = X_neurone @ poids
activation = 1 / (1 + np.exp(-z))
print(z)           # [ 1. -1.  0.]
print(activation)  # environ [0.7311 0.2689 0.5]
```

Le biais b vaut ici zéro. Une activation non linéaire permet d'exprimer davantage qu'une succession de transformations linéaires. Ces nombres ne sont pas encore des probabilités validées : les poids sont arbitraires et aucun apprentissage n'a eu lieu.

Autoévaluation

1. Si `X.shape == (50, 3)` et `w.shape == (3,)`, quelle est la forme de `X @ w` ?
2. `X * w` et `X @ w` font-ils le même calcul ?
3. Une expression NumPy sans `for` est-elle nécessairement en $O(1)$?
4. Si la dérivée de la perte est positive, faut-il augmenter ou diminuer le paramètre pour descendre localement ?
5. `stats.norm.pdf(100)` est-il une probabilité ? Que donne `stats.norm.ppf(0.95, loc=100, scale=15)` ?

Justifier chaque réponse à partir des dimensions, du calcul effectué ou de la définition utilisée.

Les réponses et leurs raisons

1. `(50,)` : un score par observation.
2. Non : `*` produit 50 lignes de 3 produits ; `@` additionne les trois produits de chaque ligne.
3. Non : il faut encore traiter les valeurs. Le gain vient notamment de l'exécution compilée et de la représentation des données.
4. Le diminuer avec un pas adapté : la mise à jour soustrait la dérivée multipliée par un pas positif.
5. `pdf` donne une densité, pas une probabilité ponctuelle. `ppf` donne ici un seuil d'environ 124,67 ms.

Ces réponses mobilisent les formes des tableaux, les calculs vectorisés, les dérivées et les lois de probabilité.

Syntaxe à retrouver sans la mémoriser

Besoin	Outil
Tableau, grille de valeurs	`np.array`, `arange`, `linspace`
Formes et orientation	`shape`, `reshape`, `.T`
Résumer par variable	`mean(axis=0)`, `std(axis=0)`
Opérations par case	`*`, `**`, `exp`, `log`
Produit matriciel	`@`
Système / moindres carrés	`linalg.solve`, `linalg.lstsq`
Minimum / intégrale	`optimize.minimize`, `integrate.quad`
Loi de probabilité	`pmf/pdf`, `cdf`, `sf`, `ppf`, `rvs`

Avant l'appel : écrire ce que l'on cherche. Après l'appel : vérifier forme, unités et résultat sur un cas connu.

Complément : une probabilité conditionnelle

$P(A \mid B)$ se lit « probabilité de A sachant que B a eu lieu ». On ne regarde plus que les situations où B est vrai :

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \qquad P(B) > 0.$$

$A \cap B$ signifie que les deux événements ont lieu.

Exemple : sur 100 commandes, 40 viennent de clients abonnés ; parmi ces 40 commandes, 10 dépassent 50 euros. La proportion de grosses commandes chez les abonnés vaut $10/40 = 25\,\%$.

L'indépendance donne $P(A \mid B) = P(A)$. En général, $P(A \mid B)$ et $P(B \mid A)$ sont différents : ils n'ont pas le même dénominateur.

Complément : Bayes et les fausses alertes

Une pièce est défectueuse avec probabilité 0,01. Un contrôle alerte dans 95 % des cas défectueux, mais aussi dans 5 % des cas sans défaut. Notons D le défaut et A l'alerte.

$$P(D \mid A) = \frac{P(A \mid D)P(D)}{P(A \mid D)P(D) + P(A \mid \overline{D})P(\overline{D})}.$$

$\overline{D}$ signifie « sans défaut ». Sur 10 000 pièces, le modèle donne en moyenne 95 alertes avec défaut et 495 alertes sans défaut.

```
p_defaut_sachant_alerte = (0.95 * 0.01) / (
    0.95 * 0.01 + 0.05 * 0.99)
print(p_defaut_sachant_alerte)  # environ 0.1610
```

Parmi les alertes, environ 16,10 % correspondent à un défaut. Ce calcul éclaire la précision d'un classifieur lorsque la classe recherchée est rare.

Complément : covariance et corrélation

La **covariance** décrit comment deux variables varient ensemble. Pour $n > 1$ observations :

$$s_{xy} = \frac{1}{n-1} \sum_i (x_i - \bar{x})(y_i - \bar{y}), \qquad \rho = \frac{s_{xy}}{s_x s_y}.$$

La **corrélation** normalise la covariance ; elle est comprise entre -1 et 1 quand les deux écarts-types sont non nuls. Elle mesure une association linéaire, pas une causalité.

```
observations = np.array([[1., 2.], [2., 1.],
                         [3., 4.], [4., 3.]])
C = np.cov(observations, rowvar=False, ddof=1)
print(C)  # environ [[1.6667, 1.], [1., 1.6667]]
print(np.corrcoef(observations, rowvar=False)[0, 1]) # 0.6
```

`rowvar=False` précise que les variables sont dans les colonnes. La diagonale de `C` contient les variances ; les autres cases sont les covariances.

Complément : directions propres et ACP

Pour une matrice C, un **vecteur propre** $v \neq 0$ vérifie $Cv = \lambda v$: l'application de C change sa longueur ou son sens, mais garde sa direction. λ est la **valeur propre** associée.

Une matrice de covariance réelle est symétrique : $C^T = C$. `linalg.eigh(C)` lui convient et renvoie les valeurs propres en ordre croissant, puis les vecteurs propres en colonnes.

```
valeurs, vecteurs = linalg.eigh(C)
print(valeurs)  # environ [0.6667 2.6667]
direction = vecteurs[:, -1]   # direction de plus forte variance
print(valeurs[-1] / valeurs.sum())  # 0.8
assert np.allclose(C @ direction, valeurs[-1] * direction)
```

L'**analyse en composantes principales (ACP)** projette les données centrées sur des directions de forte variance. Ici, la première porte 80 % de la variance totale. v et $-v$ décrivent la même direction.

Source : SciPy : linalg.eigh

Complément : l'incertitude sur une moyenne

Pour n observations indépendantes d'une population normale, de variance inconnue, un intervalle de confiance à 95 % pour la moyenne est :

$$\bar{x} \pm t_{0{,}975,n-1} \frac{s}{\sqrt{n}}.$$

s utilise une division par $n-1$; $s/\sqrt{n}$ est l'erreur standard estimée de la moyenne. $t_{0{,}975,n-1}$ est un quantile de la loi de Student à $n-1$ degrés de liberté.

```
mesures = np.array([98., 102., 99., 101., 100.])
se = stats.sem(mesures)  # écart-type avec ddof=1 / racine de n
intervalle = stats.t.interval(0.95, df=len(mesures)-1,
                              loc=mesures.mean(), scale=se)
print(intervalle)  # environ (98.04, 101.96)
```

La procédure couvrirait la vraie moyenne dans 95 % des échantillons répétés sous ces hypothèses. Cet intervalle concerne la moyenne, pas l'étendue de 95 % des observations.

Source : SciPy : erreur standard et loi de Student